

Physical Intelligence (π)[Home](#) [Research](#) [Join Us](#)

Precise Manipulation with Efficient Online RL

Published
March 19, 2026

Email
research@physicalintelligence.company

Paper
[RLT.pdf](#)

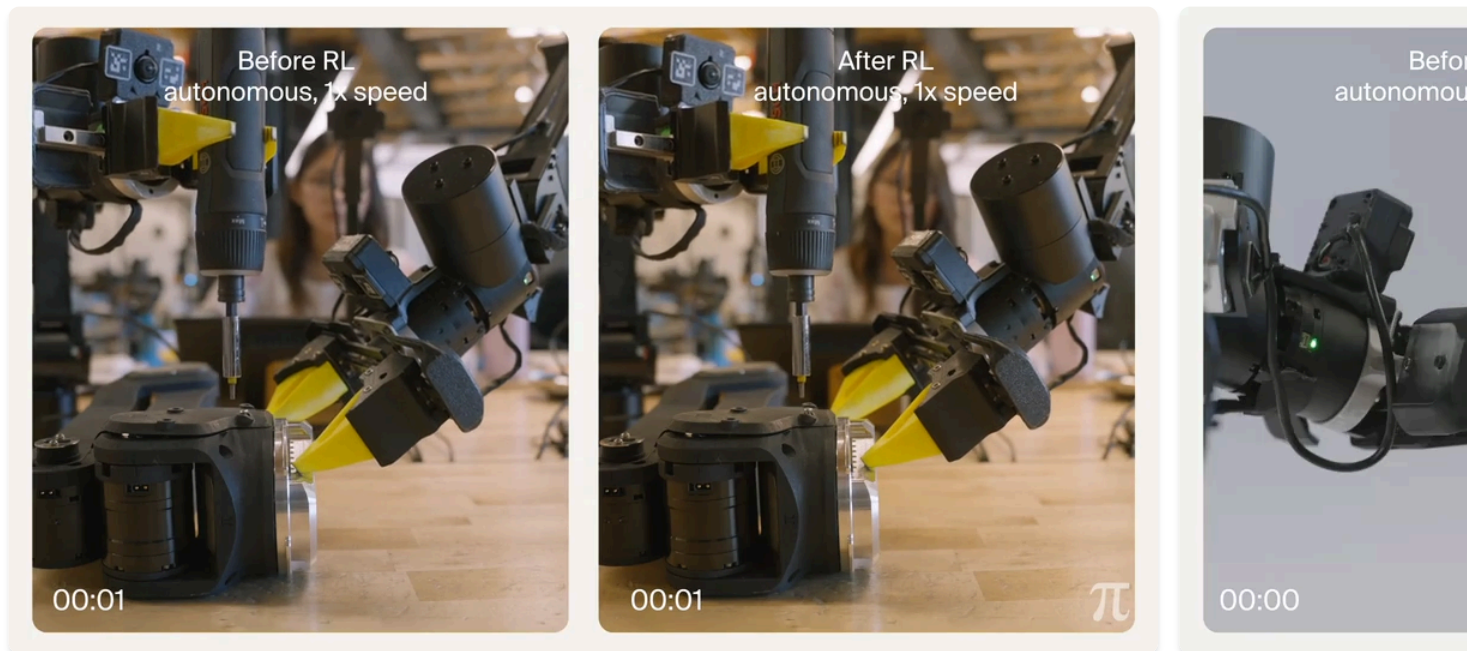


Our models can follow diverse instructions and perform a wide range of tasks, from [folding laundry](#) and [making coffee](#) to [cooking a grilled cheese sandwich](#). But for many applications, broad competence is not enough: the hardest parts of physical tasks require precision, dexterity, and speed. Picking up a screwdriver may be easy, but aligning it with a tiny screw quickly and precisely is much harder.

Reinforcement learning (RL) is a natural way to improve such tasks: by trying, observing outcomes, and adjusting, a robot can refine behaviors that are difficult to learn from demonstrations alone. Prior robotic RL methods, including our recent [RECAP](#) algorithm, focus on broad improvement over long-horizon tasks with large-scale data collection. In this post, we describe a new method that we've developed that we call RL tokens (RLT), which instead specifically aims to improve precise and

delicate tasks that require fine-grained manipulation, learning from just minutes or hours of real-world experience.

RLT adds a special output token that provides a compact interface between the VLA and a lightweight RL policy, allowing the robot to adapt its behavior in just a few hours without fine-tuning the full model. Across four challenging manipulation tasks, RLT speeds up the most precise stages of each task by up to 3×, and can surpass the speed of human teleoperation.



Each task before and after RLT training. The model after RL training is significantly faster and more performant than the base model.

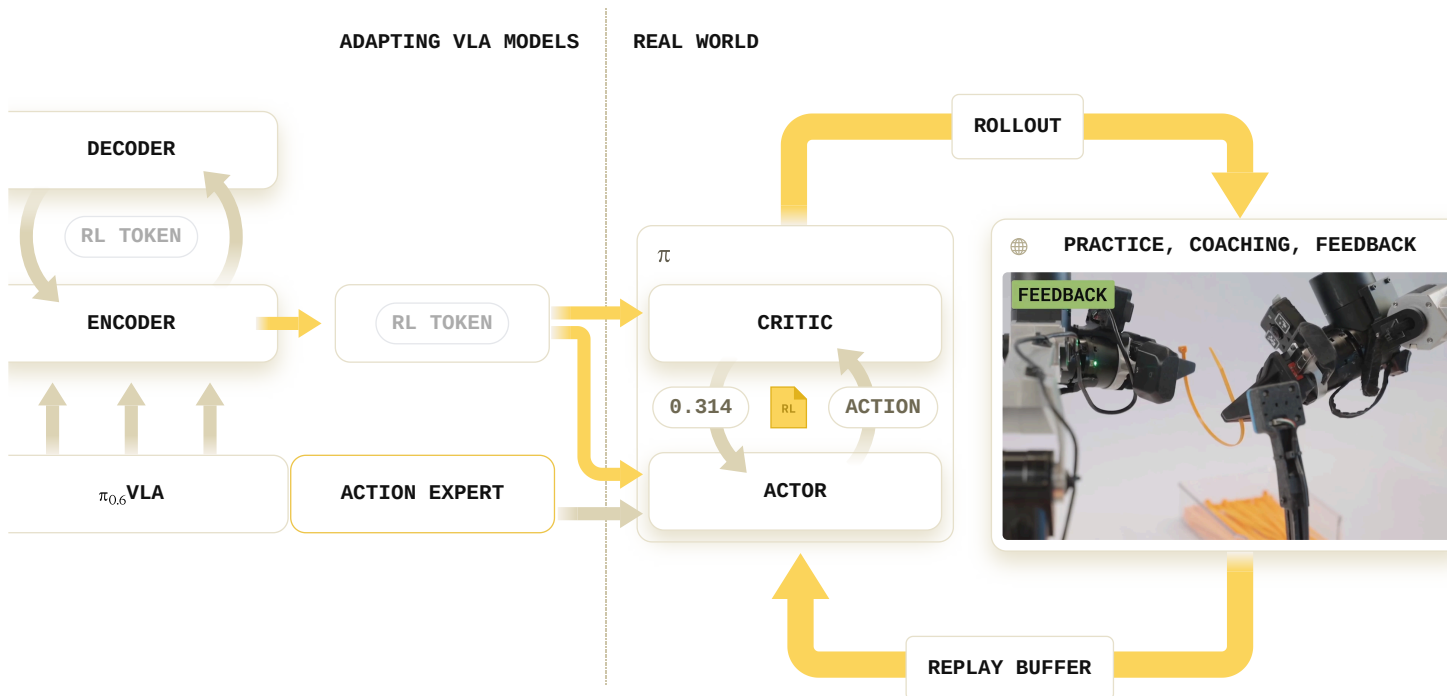
Adapting VLAs for Fast Online RL via RL Tokens

We have previously demonstrated that VLAs can learn from experience via RL, with a method we called RECAP. While RECAP focused on large-scale RL with long-horizon tasks, in many cases we would like robots to improve individual and particularly challenging stages of a specific skill quickly, with hours or even minutes of data. For example, a robot that needs to precisely use a screwdriver for an assembly can finetune just the skill of aligning the screwdriver with the screw, potentially much more quickly than it could finetune the entire VLA model. This kind of precisely targeted adaptation could even run directly during deployment.

Ideally, this improvement should occur directly on board the robot and extract as much learning signal as possible from each attempt. Training the entire VLA model to improve the task end-to-end in just a few hours presents major computational and practical challenges. Our main insight is that we can instead adapt the VLA to be amenable to RL fine-tuning with a much smaller model that can be updated in real time. We train the VLA to produce an "RL token" that can then provide a concise summary of the VLA's internal representations. This RL token is then used as the input into a much smaller model that can be trained with RL in real time.

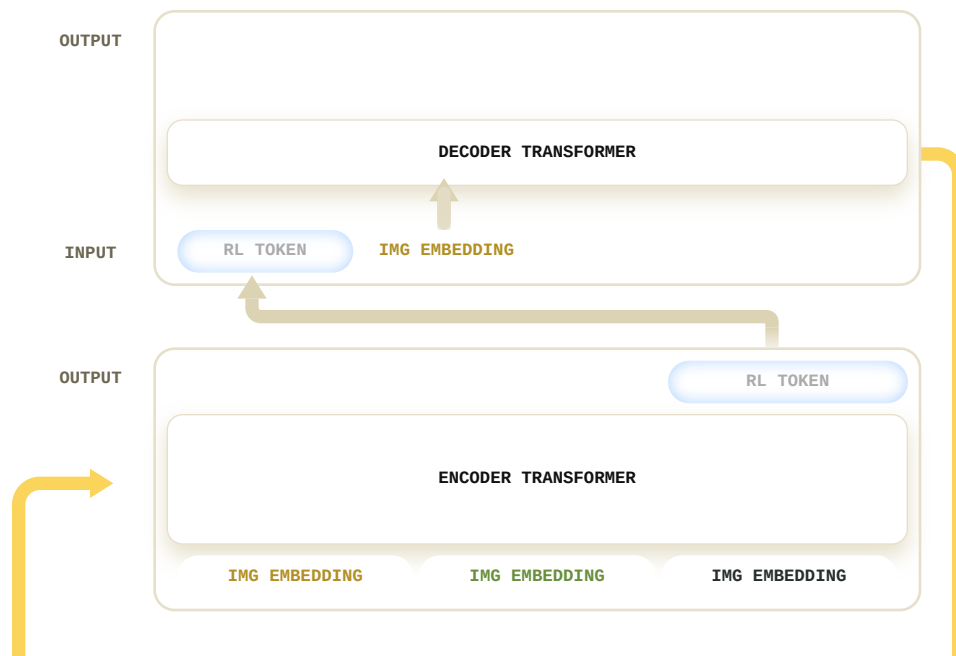
The RL token is used by an actor and critic, which are trained with a sample-efficient off-policy RL method. Because the actor and critic operate on this compact representation, they can be represented

with small networks that are trained directly on the robot, with hundreds of updates per second. This makes RL training responsive enough to improve the behavior after each attempt.



We train the VLA to provide an RL token that summarizes the VLA's internal representations and is frozen after training. This RL token is then provided to the actor and critic networks, which are trained with an efficient RL algorithm to improve the most challenging and precise phase of the task.

RLT first adapts the VLA by adding an encoder-decoder transformer that is trained to predict the model's internal embeddings through a bottleneck, leading to a compressed representation that we call the RL token. This RL token summarizes the information that the RL actor and critic need about the current observation, making it possible for even a very small actor and critic to learn to improve the model based on its own rich internal representations.



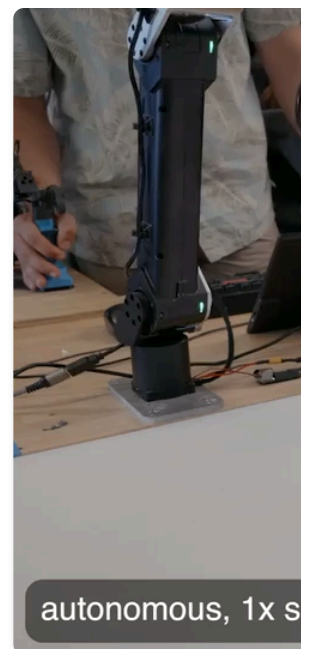


We train the VLA to produce the RL token by adding an encoder-decoder transformer. The final output from the encoder - the RL token - must retain enough information for a decoder to reconstruct the original VLA embeddings, forcing it to act as an information bottleneck. After training, the RL token provides a compact state representation for the online RL actor and critic.

With the RL token, we train small actor and critic networks with online RL with just a few hours or even minutes of robot data. Making this process maximally efficient requires a few careful design decisions. The online RL actor has to operate on the same action space as the VLA, stay grounded to the VLA's prior behavior, and learn efficiently from limited real-world data. We accomplish this as following:

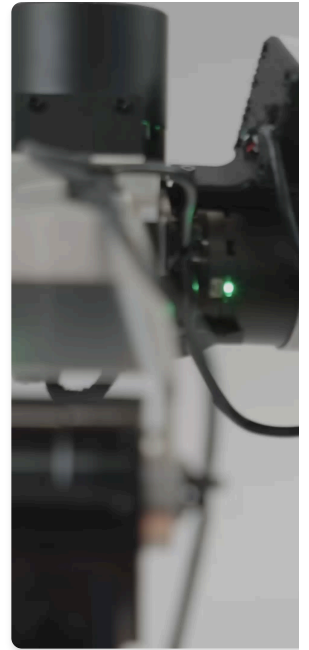
First, the RL policy predicts action chunks, *matching the action structure used by the VLA* rather than acting at individual low-level control steps. This lets the online policy adapt the same temporally extended motions that matter in our tasks. Second, the RL policy does not act from scratch: the actor receives the VLA's predicted action as input, so it learns to edit the VLA action rather than replace it entirely. We regularize the policy update toward this reference action, so the exploration stays close to the VLA when its behavior is already reasonable and deviates only when the critic identifies a better alternative. To prevent the policy from simply copying the VLA early in training, we also apply reference-action dropout, which forces the actor to maintain an independent action-generation pathway. Finally, we can optionally incorporate human interventions directly into the RL update, folding corrections back into training when the robot gets stuck or makes a mistake. These choices make online RL a reusable recipe that can be attached to a pretrained VLA across different tasks without task-specific engineering.

Succeeding in the Critical Last Millimeter of Challenging Manipulation Tasks



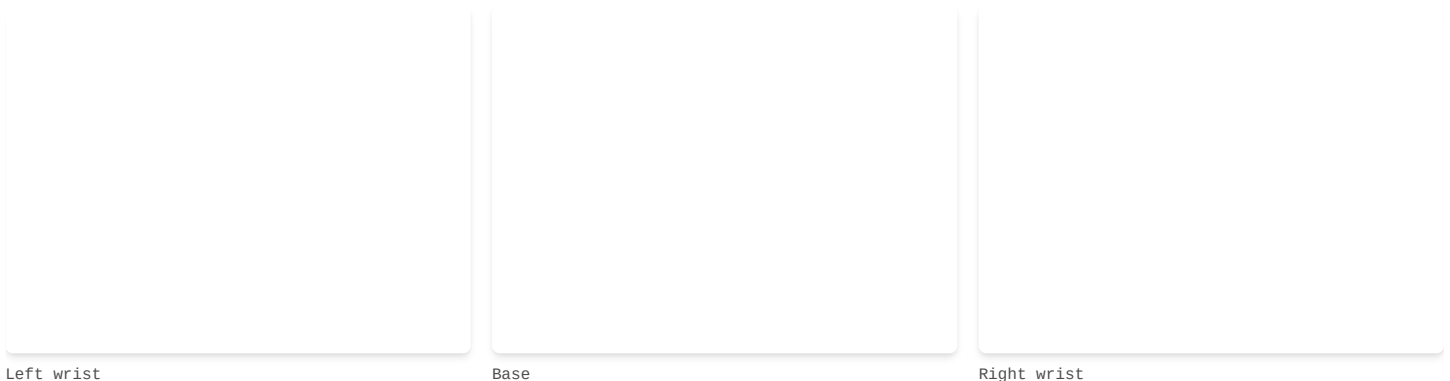
RL training improves performance on the most precise and critical part of each task.

We evaluated RLT on four challenging manipulation tasks that require high precision at critical moments: using an electric screwdriver to drive a tiny M3 screw into a robot arm, zip tie fastening, ethernet cable insertion and power-cord insertion. In each task, the generalist model can often complete the broad part of the task, but success and speed ultimately depends on a contact-rich critical phase where small errors in position, orientation, or timing lead to failure.



Small mistakes can cause each of the tasks to fail during the most precise manipulation phase. Training with RL teaches the policy to avoid such mistakes.

These tasks require considerable precision. For the screwdriver task, for example, the robot must achieve sub-millimeter precision in both position and rotation to align the screw with the screwdriver tip. Because the tip is about 10 centimeters from the grasp point, even small errors in the robot's grasp or wrist motion are amplified at the point of contact. From the robot's own wrist-camera view, these interactions are difficult to even perceive clearly.

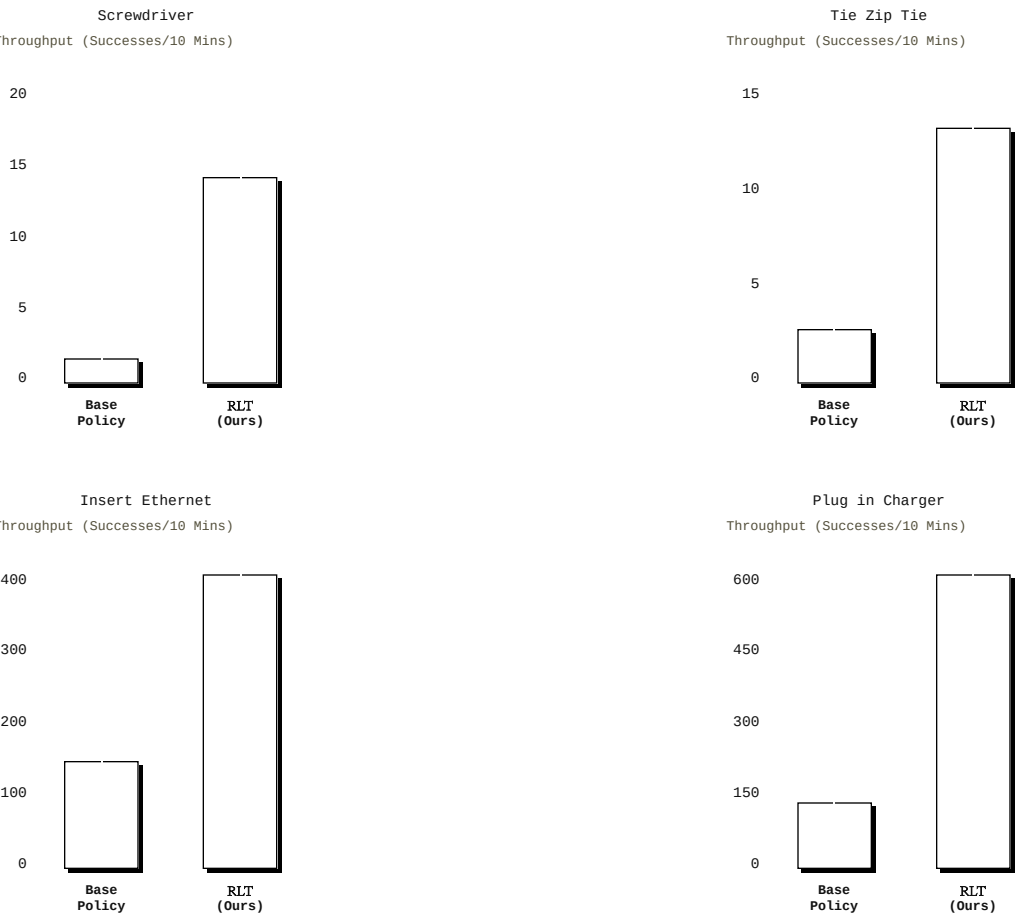


The screwdriver task from the robot's point of view, showing the base camera and both wrist cameras.

In all four tasks, the base VLA model performs well in the initial phases of the behavior—for example, picking up the screwdriver or zip tie—but struggles at the stage that demands the most precision. RLT

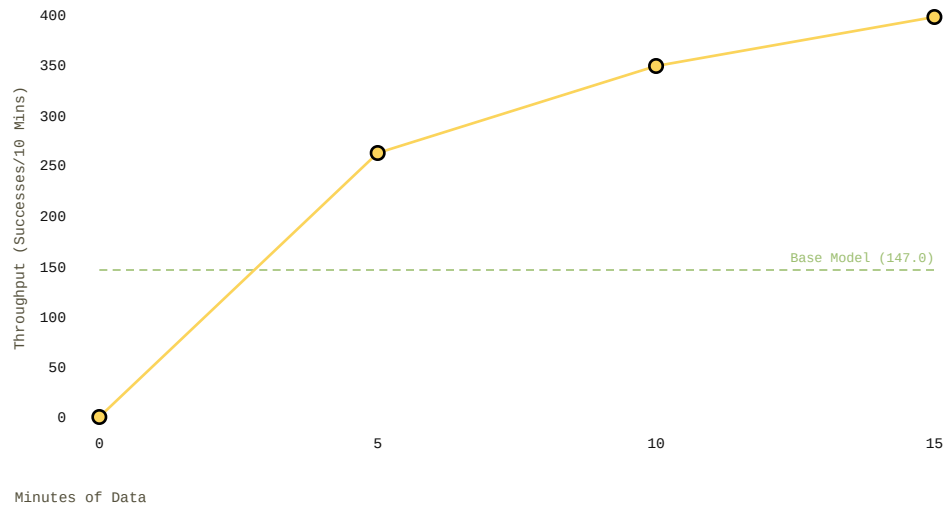
is designed exactly for this setting: rather than relearning the whole task, we use online RL to improve these critical phases. In practice, this lets the robot improve the hardest part of each behavior with as little as 15 minutes of real-world data.

We apply RLT to the critical phases of four tasks and evaluate its impact on two settings: on the short, critical phase of insertion (cord and ethernet) and on the long-horizon, higher variation task. Across all four tasks, RLT produces rapid gains in speed and success compared to the base model. The plots below show the performance of both the base model and the model after RL training, measured in terms of throughput (the number of successful task completions per 10 minutes).

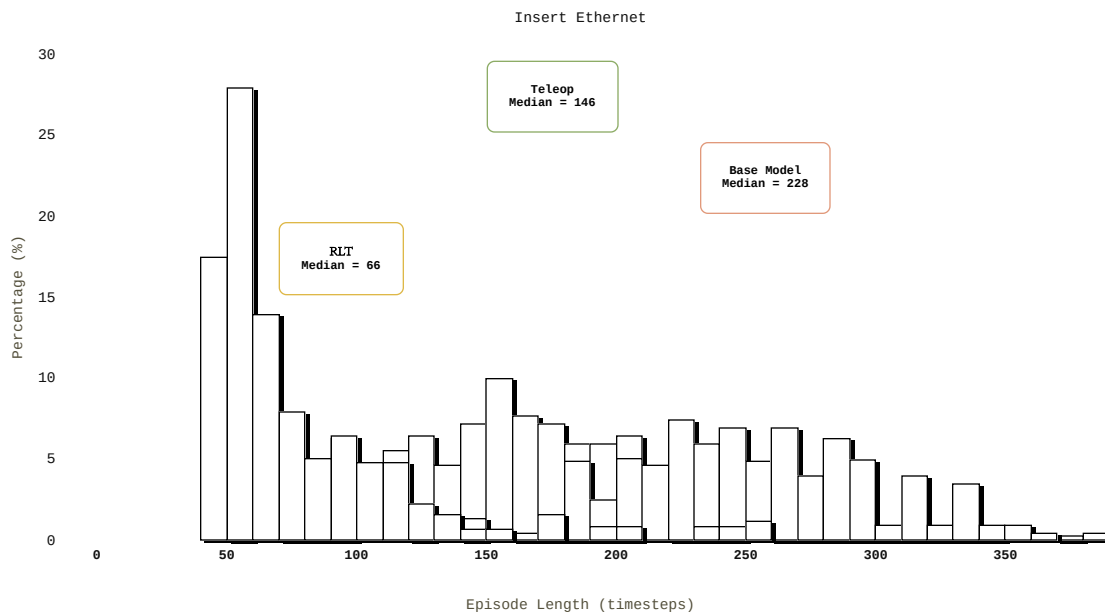


The progress plot below shows throughput improvement of RLT on the Ethernet insertion task. Training takes a total of two hours, with just 15 minutes of total robot data, with the rest of the time taken up by resets and other overhead.

INSERT ETHERNET



RLT not only improves performance over the base model, but actually results in faster execution of the task than human teleoperation on the Ethernet Task. As illustrated in the histogram below, half of the trials from the final RL policy are faster than any teleoperated demonstration in the dataset.



What's next

With RLT, we can quickly and efficiently adapt individual stages of longer tasks with online RL in real time, bringing our models closer to being able to learn directly on the job even for delicate and very precise tasks. Improving through experience will be a critical ability for real-world robotic foundation models, and future models will need to adapt at many time scales and many levels of abstraction, from the kind of fine-grained refinement of individual behaviors that we discussed here, to large-scale training of the entire model that we explored in our [previous work on RECAP](#), all the way to RL-based adaptation of higher-level reasoning capabilities, which might be informed by human feedback. Once our models can learn at all of these levels of abstraction directly from experience, they will get better and better even as they are used for practical real-world tasks. We are excited to build out this future and if you are interested in contributing to the development of this and other physical intelligence capabilities, please [get in touch](#).

